

Department of Computer Science

CS-402: Advanced Database Management Systems

Field	Details
Course	Advanced Database Management Systems
Course Code	CS-318
Instructor	Lab Instructor, HITEC University Taxila
Total Marks	100
Weightage	20% of Final Grade
Team Size	1 to 3 Students
Tech Stack	ASP.NET Core 8 (C#) with SQL Server via SSMS
Deployment	Live public URL required (Any Free of cost)
AI Tools	Permitted with mandatory documentation
Submission	GitHub repository plus live site URL plus viva

Project Title: HITEC Smart University Portal (Hi SUP)

1. Project Overview

Your task is to build and publicly deploy a full-stack web application called the HITEC Smart University Portal, or HiSUP. The portal will handle the core operations of a university campus: student records, course registration, faculty management, fee processing, library services, and result management. It is modelled on the kind of system a real institution like HITEC University Taxila would use.

The project is scoped specifically so that every topic covered in this course has a natural home inside the system. You will start development locally using SQL Server Management Studio and then migrate your database to a cloud SQL Server host for live deployment. At the time of your viva the site must be reachable by anyone on the internet.

Project:	HITEC Smart University Portal (HiSUP)
Backend:	ASP.NET Core 8 with C#, using MVC or Razor Pages
Database:	SQL Server 2019 or later, managed through SSMS
Cloud DB:	Azure SQL, Railway (MSSQL plugin), or freemssqlserver.com
Hosting:	Any platform that gives a public HTTPS URL for an ASP.NET app
Marks:	100 Weightage: 20% AI tools: allowed with documentation

2. What You Will Learn

By the time you submit this project you should be able to do all of the following without referring to notes:

1. Design a fully normalised relational database up to 3NF and BCNF, starting from a real-world problem.
2. Write stored procedures, user-defined functions, and triggers that enforce business logic at the database level.
3. Create and use views, indexes, and execution plans to improve query performance.
4. Manage transactions correctly so that concurrent users cannot corrupt data.
5. Implement proper database security using roles, row-level security policies, and column encryption.
6. Use advanced SQL features including CTEs, window functions, dynamic SQL, PIVOT, and the MERGE statement.
7. Connect a live ASP.NET Core application to SQL Server using Entity Framework Core and ADO.NET.
8. Migrate a locally developed database to a cloud SQL Server host and keep both environments in sync.
9. Deploy a complete web application with a live public URL that stays up between submission and viva.
10. Maintain a professional GitHub repository with a commit history that shows steady, incremental work.

3. Portal Modules and What Each One Requires

HiSUP is split into seven modules. Every module contributes to a single SQL Server database named HiSUP_DB. The modules are designed so that each one naturally demands a different set of database concepts; you cannot cut corners on one module without the corresponding concepts going missing from your submission.

Module 1: Database Design and Schema

Before writing any C# code, you must produce a complete database design and get it reviewed. A shaky schema that is fixed halfway through the project creates far more work than taking time to get it right at the start.

- **ER Diagram:** Draw a complete ERD covering all entities, relationships, cardinalities, and both primary and foreign keys. Export it as a PNG file and commit it to the docs folder. Tools such as dbdiagram.io or draw.io are free and work well for this.
- **Normalisation:** Document the steps to take at least three of your tables from unnormalised form through 1NF, 2NF, and 3NF. Show the functional dependencies at each stage. This documentation goes in the project report and will be discussed in the viva.
- **Minimum tables required:** Students, Faculty, Staff, Departments, Programs, Courses, Sections, Enrollments, Grades, AttendanceRecords, FeeStructure, FeePayments, LibraryItems, LibraryIssues, Hostels, HostelAllotments, ExamSchedule, Results, UserAccounts, and AuditLog. You may add more as needed.
- **Constraints:** Every table must have a primary key. Foreign keys must specify ON DELETE and ON UPDATE behaviour. Use CHECK, UNIQUE, DEFAULT, and NOT NULL constraints wherever they are genuinely meaningful, not just as decoration.

Module 2: Stored Procedures, Functions and Triggers

Business logic belongs in the database, not scattered across controller methods. The objects below are the minimum you need to build; you should add more wherever the application needs them.

- **Stored procedures, at least 15:** RegisterStudent, EnrollInCourse, ProcessFeePayment, GenerateTranscript, CalculateSemesterGPA, MarkAttendance, AllocateHostelRoom, IssueLibraryBook, ReturnLibraryBook, AddExamResult, GetStudentReport, GetFacultyWorkload, GetDepartmentEnrollment, GenerateFeeSlip, SearchCourses.
- **User-defined functions, at least 5:** fn_CalculateCGPA, fn_GetLetterGrade, fn_IsLibraryItemAvailable, fn_GetOutstandingFee, fn_GetAttendancePercentage.
- **Triggers, at least 6:** trg_AfterEnrollment to update seat count, trg_AfterGradeInsert to recalculate GPA, trg_AfterFeePayment to update balance, trg_AuditStudentUpdate to write changes to the AuditLog table, trg_PreventDuplicateEnrollment as an INSTEAD OF trigger, and trg_AfterLibraryReturn to restore the copies-available count.

Every stored procedure must include proper TRY/CATCH error handling and use RAISERROR or THROW for meaningful error messages. Input parameters must be validated inside the procedure, not only in C#.

Module 3: Views, Indexes and Query Performance

This module is about making the database fast and the queries readable. You need to show real before-and-after improvement, not just create objects and hope the examiner believes they help.

- **Views, at least 8:** vw_StudentDashboard, vw_FacultyCourseLoad, vw_DepartmentEnrollmentSummary, vw_FeeDefaulters, vw_AttendanceShortfall, vw_LibraryOverdue, vw_ExamTimetable, vw_ResultCard. At least two must be updatable views created WITH SCHEMABINDING.
- **Indexes, at least 6:** Primary key clustered indexes are created automatically. You need to add non-clustered indexes on the StudentID and CourseID columns of Enrollments, on FeePayments.PaymentDate, on LibraryIssues.ReturnDate, and on AttendanceRecords grouped by StudentID and Date. Include at least one filtered index and one covering index.
- **Query optimisation evidence:** Pick three queries that run slowly without an index. Capture the execution plan for each before adding an index, then add the index and capture it again. Put both screenshots side by side in the project report and write a short explanation of what the plan shows.
- **Full-text search:** Enable full-text indexing on the LibraryItems table covering the Title and Author columns. The library search page must use CONTAINS or FREETEXT, not a LIKE with a wildcard on both sides.

Module 4: Transactions, Concurrency and ACID

A university portal has many users acting on the same data at once. Students registering for the last seat in a course, two cashiers processing a fee payment, a faculty member submitting grades while another reads them: all of these scenarios can go wrong if transactions are not handled properly.

- **Explicit transactions:** Fee payment, course enrolment, and hostel allocation must each be wrapped in BEGIN TRANSACTION with COMMIT and ROLLBACK inside a TRY/CATCH block. The C# code that calls these procedures must also handle deadlock victim errors with a retry loop.
- **Savepoints:** Implement SAVE TRANSACTION with partial rollback in at least one multi-step workflow. Bulk result upload is a natural fit: if one student row fails validation, roll back that row and continue processing the rest rather than aborting the entire upload.
- **Isolation levels:** Pick two isolation levels, such as READ COMMITTED and SERIALIZABLE, and write a documented test that shows what concurrency problem each one prevents or allows. Include the test script and output in the project report.
- **Deadlock handling:** Write a script that reliably produces a deadlock between two sessions. Log the deadlock event to the AuditLog table. In C#, catch error 1205 and retry the operation up to three times before surfacing the error to the user.

Module 5: Security, Roles and Row-Level Security

Security in HiSUP must be enforced at the database level. An application bug or a compromised session should not give a student access to another student's grades or fee records.

- **Database roles:** Create four roles named db_student, db_faculty, db_admin, and db_finance. Grant EXECUTE on specific stored procedures only. A student role must not be able to run

SELECT directly on the Grades, FeePayments, or Enrollments tables; access is only through stored procedures.

- **Row-Level Security:** Students may only see their own rows in Enrollments, Grades, and FeePayments. Faculty may only see sections assigned to them. Implement this with a Security Policy and a Predicate Function, not with WHERE clauses in stored procedures that can be bypassed.
- **Column encryption:** The CNIC and BankAccount columns must be encrypted. Use ENCRYPTBYPASSPHRASE or Always Encrypted and demonstrate decryption through the application.
- **Audit log:** Every INSERT, UPDATE, and DELETE on the Students, FeePayments, and Grades tables must be recorded in AuditLog by a trigger. Each row in AuditLog must capture the old value, the new value, the database user, and the timestamp.

Module 6: Advanced SQL

Reports and analytics pages in HiSUP must be driven by the database, not by loops in C# that pull thousands of rows and process them in memory. If you find yourself writing a foreach loop over a large dataset in a controller, that is a sign the logic should be in a view or a query instead.

- **Common Table Expressions:** Use a recursive CTE to represent the prerequisite chain for courses. Use a regular CTE for at least one multi-step report query such as finding the top-ranked student in each department.
- **Window functions:** The result report and fee analytics pages must use RANK, DENSE_RANK, ROW_NUMBER, NTILE, LAG, LEAD, and at least one running total built with SUM OVER a partition.
- **Dynamic SQL:** Build the advanced search and filter page using sp_executesql with parameterised dynamic SQL. The query must be injection-safe; a raw string concatenation approach will lose marks.
- **PIVOT and UNPIVOT:** Generate the semester-wise attendance matrix using PIVOT so that each semester becomes a column. Include UNPIVOT as part of the same feature or in a separate report.
- **MERGE:** Use MERGE for bulk grade import. When a student already has a grade for a section, update it. When they do not, insert it. When a row is removed from the import file, delete the existing grade. All three actions in one statement.

Module 7: Backup, Restore and Moving to the Cloud

A database that only runs on your laptop is not finished. This module takes your project from local development to a live, publicly accessible state.

- **Database script:** The entire HiSUP_DB schema, including all tables, procedures, functions, triggers, views, indexes, security objects, and seed data, must be captured in a single file named database/HiSUP_DB_Script.sql. Any team member or examiner should be able to open SSMS on a fresh machine, run this file, and have a working database in under five minutes.
- **Backup plan:** Configure a full backup job and a differential backup job. Script both jobs so they can be recreated on any SQL Server instance. Include a demonstration of restoring from the backup in the project report.

- **Cloud migration:** Export the local database and import it to Azure SQL, Railway, or freemssqlserver.com. Update the production connection string in appsettings.Production.json. The live site must read from the cloud database, not a local one.
- **Secrets management:** The local connection string may go in appsettings.Development.json. The production connection string must come from an environment variable or a platform secret, never from a file committed to GitHub.

4. ADMS Concepts Checklist

All thirty items below must be present and demonstrable in the final submission. During the viva the instructor will run live queries on your cloud database to verify each one, so do not write SQL objects that exist only on your laptop.

No.	Concept	Where It Must Appear	Required
1	ER Diagram and relational schema	docs/erd.png	Yes
2	Normalisation to 3NF and BCNF	Documented in project report	Yes
3	All constraint types	Every table in HiSUP_DB	Yes
4	Stored procedures, at least 15	database/procedures/	Yes
5	User-defined functions, at least 5	database/functions/	Yes
6	AFTER triggers, at least 4	database/triggers/	Yes
7	INSTEAD OF trigger, at least 1	database/triggers/	Yes
8	Views including updatable, at least 8	database/views/	Yes
9	Non-clustered indexes, at least 4	database/indexes/	Yes
10	Filtered index and covering index	database/indexes/ with execution plan screenshots	Yes
11	Full-text search	Library search page	Yes
12	Execution plan comparison	Three before-and-after pairs in report	Yes
13	Explicit transactions with ACID	Fee, enrolment, and hostel procedures	Yes
14	SAVEPOINT and partial rollback	Bulk result upload feature	Yes
15	Isolation levels, at least 2 shown	Documented test in project report	Yes
16	Deadlock simulation and C# retry	Test script plus AuditLog entry	Yes
17	Database roles and GRANT/DENY/REVOKE	database/security/	Yes
18	Row-Level Security policy	database/security/	Yes

HITEC University Taxila Department of Computer Science			CS-402 Spring 2025 Course Project Brief
No.	Concept	Where It Must Appear	Required
19	Column encryption	CNIC and BankAccount columns	Yes
20	Audit log via triggers	AuditLog table, updated by triggers	Yes
21	Common Table Expressions	Recursive CTE for prereqs, CTE for reports	Yes
22	Window functions, at least 5 types	Result report and fee analytics pages	Yes
23	Dynamic SQL with sp_executesql	Advanced search feature	Yes
24	PIVOT and UNPIVOT	Attendance matrix report	Yes
25	MERGE statement	Bulk grade import	Yes
26	Backup and restore scripts	database/backup/	Yes
27	Cloud database migration	Live site connected to cloud DB	Yes
28	ASP.NET Core with EF Core and ADO.NET	src/HiSUP/	Yes
29	EF Core migrations	src/HiSUP/Migrations/	Yes
30	Live public deployment	Live URL in README.md	Yes

5. Web Application Requirements

Technology Stack

Layer	Technology	Notes
Backend	ASP.NET Core 8 with C#	MVC or Razor Pages
Data access	Entity Framework Core 8 and ADO.NET	EF Core for CRUD, ADO.NET for stored procedures
Local database	SQL Server 2019 or later via SSMS	Windows Auth or SQL Server Auth both fine
Cloud database	Azure SQL, Railway, or freemssql	Free tier is acceptable
Frontend	Bootstrap 5 with Razor Views	Must be responsive on mobile screen widths

Layer	Technology	Notes
Auth	ASP.NET Core Identity	Roles: Admin, Student, Faculty, Finance
Deployment	Azure App Service, Railway, or Render	Must produce a working public HTTPS URL
Source control	Git with GitHub	Continuous commits required throughout

Pages and Features

Area	Page or Feature	Database Concept It Demonstrates
Auth	Login, register, and role-based redirect	ASP.NET Identity plus database roles
Student	Student dashboard showing GPA, attendance, fees	vw_StudentDashboard view with window functions
Student	Course registration with live seat checking	Stored procedure plus transaction plus trigger
Student	Fee payment and payment history	ACID transaction and audit log trigger
Student	Transcript download as PDF	CTE and stored procedure
Faculty	Attendance marking for a full section	MERGE statement
Faculty	Grade entry and result upload	Stored procedure with trigger for auto GPA update
Faculty	Teaching workload report	Window functions with RANK
Admin	Student management and CSV bulk import	Dynamic SQL and MERGE
Admin	Department and course management	Views and stored procedures
Admin	Execution plan viewer page	sys.dm_exec_query_plan integration
Admin	Audit log viewer with filters	AuditLog table with RLS applied
Library	Book and journal search	Full-text search with CONTAINS and FREETEXT
Library	Issue and return with automatic fine	Trigger and user-defined function
Finance	Fee analytics dashboard	PIVOT and running totals with SUM OVER
Finance	Fee defaulters report	vw_FeeDefaulters view

HITEC University Taxila Department of Computer Science		CS-402 Spring 2025 Course Project Brief
Area	Page or Feature	Database Concept It Demonstrates
Reports	Semester-wise attendance matrix	PIVOT and UNPIVOT
Reports	Top students per department	Recursive CTE with DENSE_RANK

6. GitHub Repository

GitHub is the primary record of how you built this project. Commit history will be reviewed as part of the viva. A single commit of everything on the last day shows that you did not use version control properly and will receive zero marks for this component regardless of how good the code is.

Setup

11. Create a new public repository named HITEC-ADMS-HiSUP-[YourRollNo].
12. If you prefer a private repository, add the instructor's GitHub username as a collaborator on the day you create it.
13. Write a brief README.md on the first day and update it at each phase.
14. Add a .gitignore file for Visual Studio and .NET projects. The template at github.com/github/gitignore is a good starting point.
15. Never commit a real connection string, password, or API key. Use environment variables instead. GitHub scans public repositories for credentials and will alert you, and it will cost you marks.
16. Post your repository URL and live site URL on the course portal before the final deadline.

Folder Structure

```
HITEC-ADMS-HiSUP-[RollNo] /
|-- README.md                <- Live URL, team info, setup
instructions
|-- .gitignore
|-- .github/
|   |-- workflows/
|       |-- build-and-deploy.yml    <- CI/CD pipeline (bonus marks)
|-- docs/
|   |-- erd.png                <- Full ER Diagram
|   |-- schema_diagram.png
|   |-- normalization_steps.pdf    <- 1NF to 3NF documentation
|   |-- execution_plans/        <- Before and after index screenshots
|   |-- ai_usage_log.md         <- Required AI usage documentation
|   |-- project_report.pdf
|-- database/
|   |-- HiSUP_DB_Script.sql      <- Full schema plus seed data
|   |-- procedures/              <- One .sql file per stored procedure
|   |-- functions/              <- One .sql file per function
|   |-- triggers/               <- One .sql file per trigger
|   |-- views/                  <- One .sql file per view
|   |-- indexes/                <- Index creation scripts
|   |-- security/               <- Roles, permissions, RLS policies
|   |-- backup/                 <- Backup strategy and scripts
|-- src/
|   |-- HiSUP/                  <- ASP.NET Core project
|       |-- Controllers/
|       |-- Models/
|       |-- Views/
|       |-- Data/               <- DbContext and repositories
```

```
|      |-- Services/  
|      |-- Migrations/  
|      |-- appsettings.json  
|      |-- appsettings.Development.json  
|      |-- appsettings.Production.json  
|-- data/  
|   |-- seed_students.csv  
|   |-- seed_courses.csv
```

Commit Expectations

- **Minimum 30 commits** spread across the project timeline, not clustered at the end.
- **Commit after every meaningful unit of work:** a new table, a stored procedure, a working page, a fixed bug.
- **Use a short prefix in square brackets** to make the history readable.

Acceptable commit messages:

[DB] Add Enrollments table with seat-count trigger

[Proc] RegisterStudent with TRY/CATCH and output parameter

[RLS] Add row-level security policy for student grade access

[Web] Course registration page wired up to RegisterStudent procedure

[Index] Covering index on Enrollments; before and after plan added to docs

[Deploy] Production connection string moved to environment variable

These will lose marks:

update / fix / done / changes / final version

README Requirements

```
# HiSUP: HITEC Smart University Portal  
  
## Live Site  
https://hisup-[rollno].azurewebsites.net  
  
## Team  
Name | Roll No | Contribution  
-----|-----|-----  
... | ... | ...  
  
## Stack  
ASP.NET Core 8 | SQL Server 2019 | Entity Framework Core 8 | Bootstrap 5  
  
## Local Setup  
1. Run database/HiSUP_DB_Script.sql in SSMS to restore the database.  
2. Set the connection string in appsettings.Development.json.  
3. cd src/HiSUP && dotnet ef database update && dotnet run
```

```
## ADMS Concepts
Concept                | Location
-----|-----
Stored procedures (15+) | database/procedures/
Row-Level Security     | database/security/rls_policy.sql
Window functions       | database/views/vw_StudentDashboard.sql
...                    | ...

## ER Diagram
See docs/erd.png

## AI Tools Used
See docs/ai_usage_log.md
```

GitHub Actions CI/CD (Bonus 5 Marks)

Set up a workflow that builds the project automatically on every push to main. Optionally add a deployment step to Azure or Railway.

```
# .github/workflows/build-and-deploy.yml
name: Build and Deploy HiSUP
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0.x
      - run: dotnet restore src/HiSUP/HiSUP.csproj
      - run: dotnet build src/HiSUP/HiSUP.csproj --no-restore -c Release
      - run: dotnet publish src/HiSUP/HiSUP.csproj -c Release -o ./publish
```

7. Deploying the Live Site

The live URL is a graded requirement, not an optional extra. The site must be reachable at the time of the viva. If it is down during your session, you receive zero marks for the deployment component. Set up hosting early and leave enough time to troubleshoot.

Option A: Azure (Recommended)

Students can sign up for Azure for Students using their HITEC email address and receive \$100 in free credit. This is more than enough to run the project for an entire semester.

17. Register at azure.microsoft.com/free/students with your university email.
18. Create an Azure SQL Database on the free tier, which gives 32 GB of storage.
19. Create an Azure App Service on the Free F1 tier, which supports ASP.NET Core 8.
20. From Visual Studio, right-click the project and choose Publish, then select Azure App Service.
21. In the Azure portal, go to your App Service, open Configuration, and add the production connection string as an Application Setting. Do not put it in a file committed to GitHub.
22. Your live address will be [https://\[appname\].azurewebsites.net](https://[appname].azurewebsites.net).

Option B: (Any Free)

Managing Connection Strings Safely

```
// appsettings.Development.json
// This file is safe to commit because it uses Windows Auth, no password.
{
  "ConnectionStrings": {
    "HiSUP_DB":
"Server=.;Database=HiSUP_DB;Trusted_Connection=True;TrustServerCertificate=True"
  }
}

// appsettings.Production.json
// This reads from an environment variable set on the hosting platform.
// Commit this file as-is; the real value never appears in the repository.
{
  "ConnectionStrings": {
    "HiSUP_DB": ""
  }
}

// Program.cs
builder.Services.AddDbContext<HiSUPContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("HiSUP_DB")));
```

Committing a real database password to a public GitHub repository, even once, will result in a 10-mark deduction. GitHub scans public repositories automatically and will flag it. Always use environment variables or platform secrets for anything sensitive.

8. Submission Schedule

Phase	What to Submit	How	Due
Phase 1	ERD, normalised schema, and full database script committed. A brief description of each table in the README.	GitHub commit plus portal link	Week 1
Phase 2	All stored procedures, functions, and triggers working in SSMS. ASP.NET project created and connected to the local database.	GitHub commit plus portal link	Week 1
Phase 3	Views, indexes, execution plan screenshots, and full-text search complete. Student and Faculty module pages working.	GitHub commit plus portal link	Week 1
Phase 4	Transactions, RLS, encryption, audit log, and all advanced SQL features in place. Admin and Finance module pages working.	GitHub commit plus portal link	Week 2
Phase 5	Cloud database migrated. Site deployed with a working public URL. All 30 concepts demonstrable on the live site.	GitHub plus live URL plus portal	Week 2
Report	Project report as PDF uploaded to GitHub and the course portal.	GitHub plus portal	Week 2
Viva	Individual session: live site demo, SQL queries run on the cloud database, questions on the code and design decisions.	In-lab scheduled session	Week 2

Late submission: 10 marks deducted per day. Nothing accepted more than 3 days late without written approval from the instructor obtained before the deadline.
The live site must be reachable during the viva. A site that is down at that point receives zero marks for the deployment component, no exceptions.

9. Marking Breakdown

Component	What the Examiner Is Looking For	Marks
Database design	ERD completeness, normalisation steps shown, all constraint types present, 20 or more tables	10

Component	What the Examiner Is Looking For	Marks
Stored procedures and UDFs	At least 15 procedures and 5 functions, TRY/CATCH in every procedure, meaningful parameters	12
Triggers	At least 6 triggers including one INSTEAD OF, audit trigger writing to AuditLog	8
Views and indexes	At least 8 views with 2 updatable, at least 6 indexes with filtered and covering examples	8
Full-text search	CONTAINS or FREETEXT used on library, correct results returned	4
Transactions and concurrency	Explicit transactions, partial rollback with SAVEPOINT, two isolation levels documented	10
Security	Four roles with correct permissions, RLS policy working, column encryption, audit log active	10
Advanced SQL	CTEs recursive and regular, five or more window function types, dynamic SQL, PIVOT, MERGE	12
Backup and cloud migration	Backup script present, restore demonstrated in report, cloud database connected to live site	6
ASP.NET Core application	All required pages working, EF Core and ADO.NET both used, Identity auth, responsive UI	10
Live deployment	Public HTTPS URL reachable, production DB connected, no credentials in repository	5
GitHub repository	30 or more commits spread over timeline, clean structure, README current, no secrets	8
Project report	All required sections present, execution plan screenshots, AI usage log, deployment proof	5
Viva	Clear understanding of own code, can explain every decision, runs SQL on the spot	12
Bonus: GitHub Actions CI/CD	Workflow that builds and optionally deploys on push to main	+5
	Total	120

The maximum score awarded is 100. Bonus marks can offset marks lost elsewhere but cannot push the total above 100.

10. Using AI Tools

AI tools including ChatGPT, GitHub Copilot, Claude, and Gemini are permitted. Using them is not cheating. Using them without understanding what they produced and then failing to explain it in the viva is an academic problem. All AI usage must be logged in docs/ai_usage_log.md and committed to GitHub.

What AI Is Useful For

- Generating boilerplate: controller scaffolding, DbContext setup, model classes.
- Getting a starting structure for a stored procedure or trigger that you then adapt.
- Explaining SQL Server error messages and suggesting how to fix them.
- Understanding a concept such as window functions or RLS policies before you write your own version.
- Reviewing code you have already written and pointing out potential problems.
- Generating realistic test data for the seed files.

What AI Cannot Do for You

- Design the ERD. That requires understanding the problem domain, which is yours to reason through.
- Make the normalisation decisions. Showing functional dependencies in the report means you must understand them.
- Answer viva questions. The session is individual, live, and without notes.
- Replace understanding. If you cannot walk through a piece of AI-generated code line by line in the viva, it will cost you marks.

The AI Usage Log

Create docs/ai_usage_log.md and keep it up to date every time you use an AI tool. The file must contain a table in the following format:

```
# AI Usage Log

Date          | Tool          | What I asked for          | Used in
| Modified?   | What I learned
-----|-----|-----|-----
-----|-----|-----|-----
2025-02-10 | ChatGPT      | Structure for a course enrolment proc |
RegisterStudent | Yes          | How RAISERROR differs from THROW
```

```
2025-02-15 | Copilot | DbContext setup autocomplete |  
HiSUPContext.cs | Slightly | EF Core convention for composite FK  
2025-02-20 | Claude | Explanation of Row-Level Security | Reference  
only | N/A | Predicate function syntax
```

The log is checked during the viva. If a section of your code was AI-generated, you must still be able to explain every line. Typical questions include: walk me through this stored procedure from top to bottom, or tell me what happens to the data if this trigger is dropped.

11. Project Report

Submit the file docs/project_report.pdf before the final deadline. Maximum 40 pages not counting appendices. The report must cover:

23. Title page with course name, student names, roll numbers, GitHub URL, and live site URL.
24. One-page summary of what HiSUP does and the technology stack used.
25. Database design section with the full ERD and the normalisation steps for at least three tables showing functional dependencies at each stage.
26. For each of the 30 concepts in the checklist: where it is implemented, why it is needed there, a key SQL or code snippet, and a screenshot of it working.
27. Three execution plan comparisons, each showing the plan before and after an index was added, with a written explanation of what changed.
28. A role permissions table and a written explanation of how the RLS policy and column encryption work.
29. A walkthrough of one complete transaction scenario from the user action in the browser through to the COMMIT in SQL Server, showing how each ACID property is maintained.
30. Deployment section with screenshots of the cloud database, the connection string configuration, and the live site URL.
31. GitHub section with a screenshot of the commit history.
32. AI usage summary covering how tools were used, what was learned, and what changes were made to any AI-generated output before it was committed.
33. At least four real challenges encountered during the project and how each one was resolved.
34. Half a page reflecting on what this project taught you that lectures alone did not.

12. Getting Started

Work through these steps in order on the first day. Getting the infrastructure right at the start saves a lot of trouble later.

Step 1: Create the GitHub Repository

```
# Create the repository on GitHub, then clone it locally:
git clone https://github.com/[username]/HITEC-ADMS-HiSUP-[RollNo].git
cd HITEC-ADMS-HiSUP-[RollNo]

mkdir -p docs database/procedures database/functions database/triggers
mkdir -p database/views database/indexes database/security database/backup data

git add .
git commit -m "[Setup] Initialise repository and folder structure"
git push origin main
```

Step 2: Create the ASP.NET Core Project

```
dotnet new mvc -n HiSUP -o src/HiSUP
cd src/HiSUP

dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add package Microsoft.AspNetCore.Identity.UI

cd ../../..
git add .
git commit -m "[Setup] Create ASP.NET Core MVC project with EF Core packages"
git push
```

Step 3: Create the Database in SSMS

```
-- Run this in SSMS to create the database and your first table:
CREATE DATABASE HiSUP_DB;
GO
USE HiSUP_DB;
GO

CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY IDENTITY(1,1),
    DeptName NVARCHAR(100) NOT NULL UNIQUE,
    DeptCode NVARCHAR(10) NOT NULL UNIQUE,
    EstablishedYear INT CHECK (EstablishedYear >= 1990),
    CreatedAt DATETIME DEFAULT GETDATE()
);
GO
```

```
-- Copy this script into database/HiSUP_DB_Script.sql and commit it:
git add database/HiSUP_DB_Script.sql
git commit -m "[DB] Add Departments table with constraints"
```

Step 4: Connect ASP.NET to SQL Server

```
// appsettings.Development.json
{
  "ConnectionStrings": {
    "HiSUP_DB":
"Server=.;Database=HiSUP_DB;Trusted_Connection=True;TrustServerCertificate=True"
  }
}

// Program.cs
builder.Services.AddDbContext<HiSUPContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("HiSUP_DB")));

// Run the first migration:
dotnet ef migrations add InitialCreate
dotnet ef database update
dotnet run

git add . && git commit -m "[DB] Connect to HiSUP_DB; first EF Core migration"
```

Step 5: Write Your First Stored Procedure

```
-- database/procedures/RegisterStudent.sql
CREATE OR ALTER PROCEDURE RegisterStudent
    @FirstName NVARCHAR(50),
    @LastName NVARCHAR(50),
    @Email NVARCHAR(100),
    @DeptID INT,
    @NewStudentID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
            INSERT INTO Students (FirstName, LastName, Email, DepartmentID)
            VALUES (@FirstName, @LastName, @Email, @DeptID);
            SET @NewStudentID = SCOPE_IDENTITY();
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

GO

```
git add database/procedures/RegisterStudent.sql
git commit -m "[Proc] RegisterStudent with transaction and error handling"
```

13. Preparing for the Viva

The viva is 15 marks and is conducted individually. It does not matter how large your team is; each person is examined separately on the full project. The session will run roughly as follows:

- The instructor opens your GitHub repository and scrolls through the commit history.
- The instructor opens your live site and tests core features: registering a student, enrolling in a course, making a payment, searching the library.
- SSMS is opened against your cloud database. The instructor runs queries directly.
- You may be asked to write a new query, modify an existing stored procedure, or add an index on the spot.
- If the AI usage log shows significant AI involvement in a section, you will be asked to explain it in detail.

Questions to prepare for:

35. Show me the functional dependencies for the FeePayments table and explain why it is in 3NF.
36. What happens to a student's GPA if someone deletes a row from Grades directly without using your stored procedure?
37. Open the execution plan for this query. What does it show, and which index is responsible for the improvement?
38. What isolation level does your fee payment transaction use? Why that one and not SERIALIZABLE?
39. Log in as a student account and try to run `SELECT * FROM Grades` in SSMS. What error do you get and why?
40. Walk me through the MERGE statement in your bulk grade import, line by line.
41. Your cloud database is unreachable right now. Show me your backup restore script and talk me through the steps to bring it back.
42. This section of code was in your AI usage log. Explain what it does and what you changed from what the AI originally gave you.

14. Academic Integrity

Copying another student's code or database is plagiarism: zero for both parties.
Using AI without documenting it and then being unable to explain the code in viva

is treated as undocumented AI usage: 20-mark deduction.
Committing a real database password to GitHub: 10-mark deduction.
Single bulk commit at the end: 0 marks for the GitHub component.
Live site down during viva: 0 marks for the deployment component.

You are welcome to read the SQL Server documentation, watch tutorials, discuss concepts with classmates, and use AI tools. The expectation is that the submitted work reflects your own understanding, that you can explain every part of it under questioning, and that any assistance received is acknowledged.

15. Recommended Resources

Books

- **Elmasri and Navathe:** Fundamentals of Database Systems, 7th edition. The main textbook for database theory and normalisation.
- **Itzik Ben-Gan:** T-SQL Fundamentals and T-SQL Querying. The best practical references for SQL Server.
- **Andrew Lock:** ASP.NET Core in Action, 3rd edition. Covers everything you need for the web application side.

Online

- learn.microsoft.com/sql/: official SQL Server documentation and T-SQL reference
- learn.microsoft.com/aspnet/core/: official ASP.NET Core 8 documentation
- use-the-index-luke.com: a visual and accessible guide to indexes and query optimisation
- dbdiagram.io: free browser-based ERD and schema design tool
- railway.app/docs: Railway deployment guide for .NET applications
- learn.microsoft.com/azure/app-service/: Azure App Service deployment guide

Tools

- **SQL Server Management Studio (SSMS):** free download from Microsoft, required for local development
- **Visual Studio 2022 Community Edition:** free, the most productive IDE for ASP.NET Core and SQL Server together
- **Azure Data Studio:** lighter alternative to SSMS, cross-platform, useful for working on macOS
- **Postman:** for testing any API endpoints your application exposes
- **dbdiagram.io or draw.io:** for drawing and exporting the ER diagram

HITEC University Taxila Department of Computer Science		CS-402 Spring 2025 Course Project Brief
Lab Instructor Department of Computer Science HITEC University Taxila	CS-318: Advanced Database Management Systems Spring 2025 Total Marks: 100 Weightage: 20%	